Best Practices: Versioning, Branching & Version Display

Extracted from meeting with DevOps expert Marko (2026-02-25)

1. Semantic Versioning Strategy

Version Number Format: MAJOR.MINOR.PATCH

- **MAJOR** (1st number): Breaking changes, incompatible API changes
- **MINOR** (2nd number): New functionality, backward compatible
- **PATCH** (3rd number): Bug fixes, quick updates

Key Principles

- Use semantic versioning consistently across all microservices
- Every version gets its own artifact stored in artifact storage (GCP buckets, artifact registries)
- Artifacts enable easy rollbacks if issues are discovered in testing or production
- Avoid using 'latest' tags - they break deployment systems

2. System vs Service Versioning

Service Level Versioning

- Each component maintains its own version:
  - Backend: separate artifact storage
  - Frontend: separate artifact storage
  - TMS Bridge: separate artifact storage
  - Cloud Functions: separate artifact storage

System Level Versioning

- Group specific versions of services into a **system version**
- Example: System v2.3.1 might include:
  - Backend v1.5.2

    - Frontend v2.1.0
    - TMS Bridge v1.3.4

Benefits

- Simplifies customer deployments
- Enables system-wide rollbacks
- Clear communication of what's deployed where

3. Uniform Tagging Strategy

Critical Requirements

- **Agree on ONE tagging convention** across all microservices
- Document the convention clearly
- Ensure team-wide adherence
- Inconsistent tagging WILL break the deployment system

Tagging Rules

- Tags created ONLY after pull requests are merged into main or release branches
- Ensures only stable features are deployed
- Minor and patch versions managed according to branching strategy

4. Branching Strategy

Developer Workflow

1. Feature development on feature branches
2. Pull request for review
3. Merge to main/release branch triggers pipeline
4. Pipeline creates tag and builds artifact
5. Artifact stored with version tag

Best Practices

- Define and document branching strategy clearly
- Use branching strategies to trigger pipeline automation

- Only merged features get tagged and deployed
- No tags on feature branches

5. CI/CD Pipeline Design

Pipeline Responsibilities

Pipelines should handle:

- Building artifacts
- Running tests (integration/unit)
- Tagging versions
- Pushing artifacts to storage
- Deploying artifacts

Key Principles

- **DO**: Use pipelines for versioning and deployment logic
- **DON'T**: Create dedicated microservices for version management
- Leverage existing tools (Git, Azure DevOps, GCP Cloud Build)
- Separate pipelines based on triggers and purpose

Pipeline Triggers

- On pull request: Run tests
- On merge to main: Build, tag, store artifact
- On release branch: Deploy to specific environment

Complexity Management

- Proper branching + trigger configuration = manageable pipelines
- No excessive manual intervention required
- Automation reduces human error

6. Displaying Version Information in Frontend

Problem

How to dynamically display current system and service versions in the UI?

Recommended Solution: Database Storage

During deployment:

1. Deployment pipeline writes version information to database table
2. Store both service versions and system version
3. Include deployment timestamp, environment info

At runtime:

1. Frontend queries database for current versions
2. Backend exposes API endpoint to retrieve version info
3. Display in UI (e.g., footer, about page, admin panel)

Database Schema Example

```
CREATE TABLE deployment_versions (
  id SERIAL PRIMARY KEY,
  environment VARCHAR(50),  -- e.g., 'production', 'staging'
  system_version VARCHAR(20),
  backend_version VARCHAR(20),
  frontend_version VARCHAR(20),
  tms_bridge_version VARCHAR(20),
  cloud_functions_version VARCHAR(20),
  deployed_at TIMESTAMP,
  deployed_by VARCHAR(100)
);
```

Alternative Approach: Deployment Hooks

- Use deployment hooks to fetch from artifact registry
- More complex, less reliable
- Database approach preferred for simplicity

Implementation Notes

- **DON'T**: Hardcode version in frontend
- **DON'T**: Couple version display to frontend build
- **DO**: Make it dynamic and environment-aware
- **DO**: Store version info during deployment process

7. Team Workflow & Governance

Documentation Requirements

Create and maintain documentation for:

- Branching strategy
- Tagging strategy
- Pipeline processes
- Version numbering conventions

Team Alignment

- All developers must follow agreed workflow
- Dedicated DevOps engineer to enforce standards
- Clear point of contact for workflow questions
- Regular reviews of adherence

Ownership

- Assign dedicated person/team for DevOps oversight
- Ensure strategies are integrated into team workflow
- Maintain documentation as process evolves

8. Implementation Checklist

- [ ] Define semantic versioning convention
- [ ] Document branching strategy
- [ ] Set up artifact storage for each component
- [ ] Configure CI/CD pipelines with proper triggers
- [ ] Implement tagging rules (only on merged PRs)

- [ ] Create database table for version information
- [ ] Update deployment pipeline to write versions to DB
- [ ] Create API endpoint for version retrieval
- [ ] Add version display to frontend UI
- [ ] Train team on workflow
- [ ] Assign DevOps ownership
- [ ] Review and iterate

Key Takeaways

1. **Consistency is critical** - One versioning strategy across everything
2. **Automation over manual process** - Use pipelines, not microservices
3. **Version information is deployment metadata** - Store it during deployment
4. **Simplicity over complexity** - Use existing tools effectively
5. **Documentation and training** - Ensure team-wide understanding and adherence